DOCUMENTATION AND FURTHER EXPLANATIONS

Alexandru-Gabriel Brabete
alexandrubrabete@yahoo.com

PROBLEM 1
1.1

1. MANAGEMENT LEVEL (MANAGEMENT & STAFF)

<u>CoffeeShop Class</u>

It is the class that groups all resources.

- **Attributes:**
  - **shopId:** int – Unique location identifier.
  - **name:** String – The name of the café.
  - **location:** String – Physical address.
- **Methods:**
  - **addBarista(b: Barista): void** – Registers a new employee to the team.
  - **createOrder(c: Customer): Order** – Initiates a new transaction for a customer.
- **Relationships:**
  - **1 : N with** Barista: Manage a list of employees.
  - **1 : N with Order**: Centralizes all sales made in that location.

<u>Barista Class</u>

It represents the staff who serve customers.

- **Attributes:**
  - **baristaId:** int – Employee code.
  - **name:** String – Full name.
- **Methods:**
  - **prepareOrder(o: Order): void** – Marks the start of preparing an order.
- **Relationships:**
  - **N : 1 with CoffeeShop**: Each barista is subordinate to a unit.
  - **1 : N with Order**: Associated with each order it prepares.

2. LOYALTY PROGRAM

<u>Customer Class (Abstract)</u>

Define the overall behavior of a loyal customer.

- **Attributes:**
  - **customerId:** int – Unique member ID.
  - **name:** String – The name of the client.
  - **loyaltyPoints:** int – Current points balance.
- **Methods:**
  - ***calculatedPoints(amount: double): int*** *(Abstract)* – Calculates the points earned.
  - **addPoints(points: int): void** – Increments the points balance.
  - **redeemPoints(points: int): void** – Subtracts points to get free drink.
- **Relationships:**
  - **1 : N with Order**: A customer can have an entire purchase history.

<u>Subclasele RegularMember & GoldMember</u>

- **Inheritance:** From Customer.
- **Specific:** Implements the calculated Points method differently.
  - RegularMember: 1 point / 1 euro
  - GoldMember: 2 points / 1 euro

3. TRANSACTIONAL SYSTEM (ORDERS)

<u>Order Class</u>

The document that records what was bought and by whom.

- **Attributes:**
  - **orderId:** int – The unique number of the order.
  - **timestamp:** DateTime – The exact date and time of your purchase.
  - **totalPrice:** double – The total payout amount.
- **Methods:**
  - **calculatedTotal(): double** – Calculates the sum of all drinks in the list.
  - **addDrink(d: Drink): void** – Adds a custom drink to your order.
- **Relationships:**
  - **N : 1 with Customer**: The order is assigned to a customer.
  - **N : 1 with Barista**: The order is linked to the person who prepares it.

- **1 : N (Composition) with Drink**: The command contains multiple drink-type objects.

4. PRODUCT STRUCTURE (DRINKS & CUSTOMIZATION)

Class Drink (Abstract)

- **Attributes:**
  - **drinkId:** int – Internal product code.
  - **size:** Size – A reference to the chosen size type.
  - **basePrice:** double – The starting price according to size.
- **Methods:**
  - ***calculatedPrice(): double*** – Base price + extras.
  - **addExtra(s: Extra): void** – Add a topping or syrup.
- **Relationships:**
  - **N : 1 with Size**: Each drink has a preset size.
  - **1 : N with Extra**: A drink can be customized with a list of extras.
- **Subclasses:** Espresso, Latte, Cappuccino (Each with its own specific price).

Extra Class

- **Attributes:**
  - **extraId:** int – topping ID (ex: 1 for Caramel).
  - **name:** String – The name (Vanilla, Extra Shot, etc.).
  - **price:** double – The fixed cost of this markup.
- **Relationships:**
  - **N :M with Drink**: A type of Extra can be added to several different drinks.

Enum Size

- **Values:** Small, Medium, Large.
- **Role:** Limits options for drink size, ensuring data consistency.

Summary of the logical flow in the classes:

When Sarah wants to see the performance, she queries a **CoffeeShop** object, which returns her list of **Orders**. Each order tells her who the **Barista was**, which **Customer** benefited from points and which list of **Drinks** (with **related Extras) was sold, at what time and at what price.**

1.2

DATA ARCHITECTURE (SQL IMPLEMENTATION)

The code is organized to ensure data integrity (through constraints and foreign keys) and automation of workflows.

1. Table structure and hierarchy

The system is built on three main pillars:

- **Location management (COFFEESHOPS, BARISTAS):** Manages physical infrastructure and staff. The relationship is 1:N, it ensures that every sale is traceable back to the store and the employee who made it.
- **Product catalog (BEVERAGE_CATEGORIES, BEVERAGES, EXTRAS):** Implements a flexible structure. Instead of fixed prices in the code, BEVERAGES allows you to define the base price according to size (Small, Medium, Large), and the EXTRAS table allows you to add dynamic additional costs.
- **Trading engine (SALES, SALE_DETAILS, SALE_DETAIL_EXTRAS):** Uses a granular structure:
    - SALES: The transaction header (date, customer, total amount).
    - SALE_DETAILS: Individual products in the receipt.
    - SALE_DETAIL_EXTRAS: The specific options applied to each individual drink.

2. Automation and business logic

Programmable objects that reduce human error:

- **Automatic calculation of points (udf_CalculatePoints & trg_UpdatePoints)**:
    - **The function** encapsulates the business rule: for Gold and for Regular.
    - **The trigger** ensures that the customer's balance is automatically updated as soon as a sale is inserted into the SALES table. The customer does not have to "ask" for the points; The system provides them instantly.
- **Stored Sale Procedure (sp_CreateSale):**
    - This takes the ID of the barista, customer, and drink, plus a list of extras.
    - It calculates the total price (Base + Sum of extras) using STRING_SPLIT and inserts the data simultaneously into three different tables, maintaining the integrity of the transaction.

3. Analysis and Reporting (Reporting Layer)

The code includes quick data visualization tools:

- **vSalesReport View**: Unify 6 different tables into one easy-to-read view. Sarah can see who bought, what he/she bought, from whom, and in what location, without writing complex joins.
- **Tabular functions (ufn_SalesByCustomer, ufn_SalesByBarista):** Allow instant performance filtering. For example, you can see all the sales made by Andrei (Barista 1) by simply SELECT.

The code allows:

1. **Data validation**: It uses CHECK constraints which prevents the entry of erroneous data.
2. **Traceability**: By linking each sale to a BaristaID and a ShopID, Sarah can generate performance reports per employee or location.
3. **Scalability**: If Sarah decides to add "Pumpkin syrup", she just needs to insert a row into the EXTRAS table, and the sp_CreateSale procedure will automatically recognize it without any code changes.

PROBLEM 2
2.1; 2.2; 2.3; 2.4

SIEMARKET ARCHITECTURE DOCUMENTATION (C#)

The system is designed to manage the flow of orders from instantiating individual products to generating store-level statistics.

2.1 Definition of classes and data structure

The system is based on three main entities that collaborate to process an order:

- **OrderItem**: Represents a line within an order.
  - **Attributes**:
    - **ProductName:** The name of the product (set to private to ensure immutability after creation).
    - **OrderedQuantity:** The number of units purchased.
    - **PricePerUnit:** Unit price (decimal for financial accuracy).
  - **Methods**: CalculatedItemTotal() – returns the raw value of that line
- **CustomerOrder**: Manages the relationship between a customer and their products.

    - **Attributes**: CustomerName, OrderItems (a list of OrderItem objects).
    - **Methods**: CalculatedFinalPrice() – centralizes the price logic of the entire order.
- **OrderStatistics**: A class of (static) utilities for analyzing data globally (over all orders).

2.2 Logic of discount (Final price)

The implementation complies with the business rule imposed by SieMarket for high-volume orders.

Calculation algorithm:

1. The system iterates through the OrderItems list and calculates the gross amount.
2. **Implementation**: decimal discountValue = TotalPrice * 0.10m; followed by subtracting it from the total.

2.3 Identification of the "star" customer (Top Spender)

The GetStarCustomer method uses a Dictionary data structure to aggregate customer spend that can appear in multiple different orders.

- **Process**:
    1. Create a dictionary where the key is the customer's name and the value is the total spent.
    2. Go through the order list and update the value for each customer (customerTotals[customerName] += orderValue).
    3. Finally, extract the key (StarCustomer) that has the maximum value associated with it.

2.4 Product Popularity Analysis (Bonus)

The GetProductTotals method solves a common problem: **data normalization**.

- **Case-Insensitivity**: Use ToLower() to ensure that "Mouse" and "mouse" are counted as the same product.
- **Aggregation**: Use a dictionary to sum the OrderedQuantity property for each unique product encountered in all processed orders.

Technical remarks:

1. **Encapsulation**: Using private set in OrderItem prevents accidental changes to a product's details (such as name) after it has already been added to an order.
2. **Data integrity**: Using the decimal type instead of float or double guarantees that no rounding errors will occur when calculating the 10% discount.
3. **Extensibility**: The OrderStatistics class being separated from the order logic allows the addition of new reporting functions (e.g. the most profitable product) without changing the basic structure of the order.